IoT-Based HVAC Temperature and Humidity Monitoring System — Project Report

ASSIGNMENT – 3

TEAM MEMBERS

- **JAGADEEP - 24MER018**
- **JEEVE P - 24MER019**
- **JOGINDER S - 24MER021**
- **RAGUL R -24MER046**

SMART IRRIGATION AND WATER MANAGEMENT SYSTEM

1. Project Overview

Introduction

Agriculture requires a significant amount of water, and conventional irrigation methods often result in unnecessary water consumption due to over-irrigation or improper scheduling. The Smart Irrigation and Water Management System is an IoT-based automated irrigation system designed to provide water to plants according to the moisture condition of the soil.

The system uses a soil moisture sensor to continuously monitor the moisture level of the soil. An ESP8266 NodeMCU acts as the main controller and processes the sensor information. A DHT11 sensor is also used to measure environmental temperature and humidity. Based on the soil moisture level, the controller automatically operates a relay, which switches the water pump ON or OFF.

The main objective is to reduce water wastage, minimize manual intervention, and provide efficient irrigation to plants.

Main Features

- Automatic irrigation based on soil moisture.
- Continuous soil moisture monitoring.
- Temperature and humidity monitoring using DHT11.
- Automatic water pump control.
- Relay-based pump switching.
- Reduced water wastage.
- Reduced manual intervention.
- IoT-ready system using ESP8266.

2. Problem Statement

Traditional HVAC systems often operate without continuous and accurate monitoring of indoor environmental conditions such as temperature and humidity. In many buildings, HVAC systems are controlled based on fixed settings or manual adjustments, which may not accurately represent the actual conditions of different rooms or zones. This can result in overheating, overcooling, excessive humidity, uncomfortable indoor environments, and unnecessary operation of air-conditioning equipment.

Another major problem is the lack of real-time monitoring and centralized data collection. Facility operators may not be able to continuously observe environmental conditions or identify abnormal changes at the right time. Without proper monitoring, HVAC equipment may continue operating even when the desired temperature has already been reached, leading to unnecessary energy consumption and increased operating costs. Similarly, high humidity levels

may remain unnoticed, potentially affecting indoor air quality, equipment performance, and occupant comfort.

Existing systems may also have limited remote monitoring capabilities, making it difficult for users to access HVAC information when they are away from the building. The lack of historical temperature and humidity data further makes it difficult to evaluate HVAC performance, compare energy usage patterns, detect inefficient operation, and plan preventive maintenance.

Therefore, an IoT-based HVAC Temperature and Humidity Monitoring System is required to provide continuous and reliable monitoring of indoor environmental conditions. The proposed system will collect temperature and humidity data using sensors, process the information through an ESP32 microcontroller, and transmit it through Wi-Fi to an IoT dashboard. The system will provide real-time visualization, data logging, threshold-based alerts, and remote monitoring. The collected data can also be analyzed to identify HVAC inefficiencies and support better energy management, improved occupant comfort, and more effective operation of HVAC systems.

- Unnecessary operation of water pumps.
- Increased electricity consumption.
- Requirement for continuous human monitoring.

Therefore, an automated system is required to monitor soil conditions and provide water only when it is necessary.

3. Proposed Solution

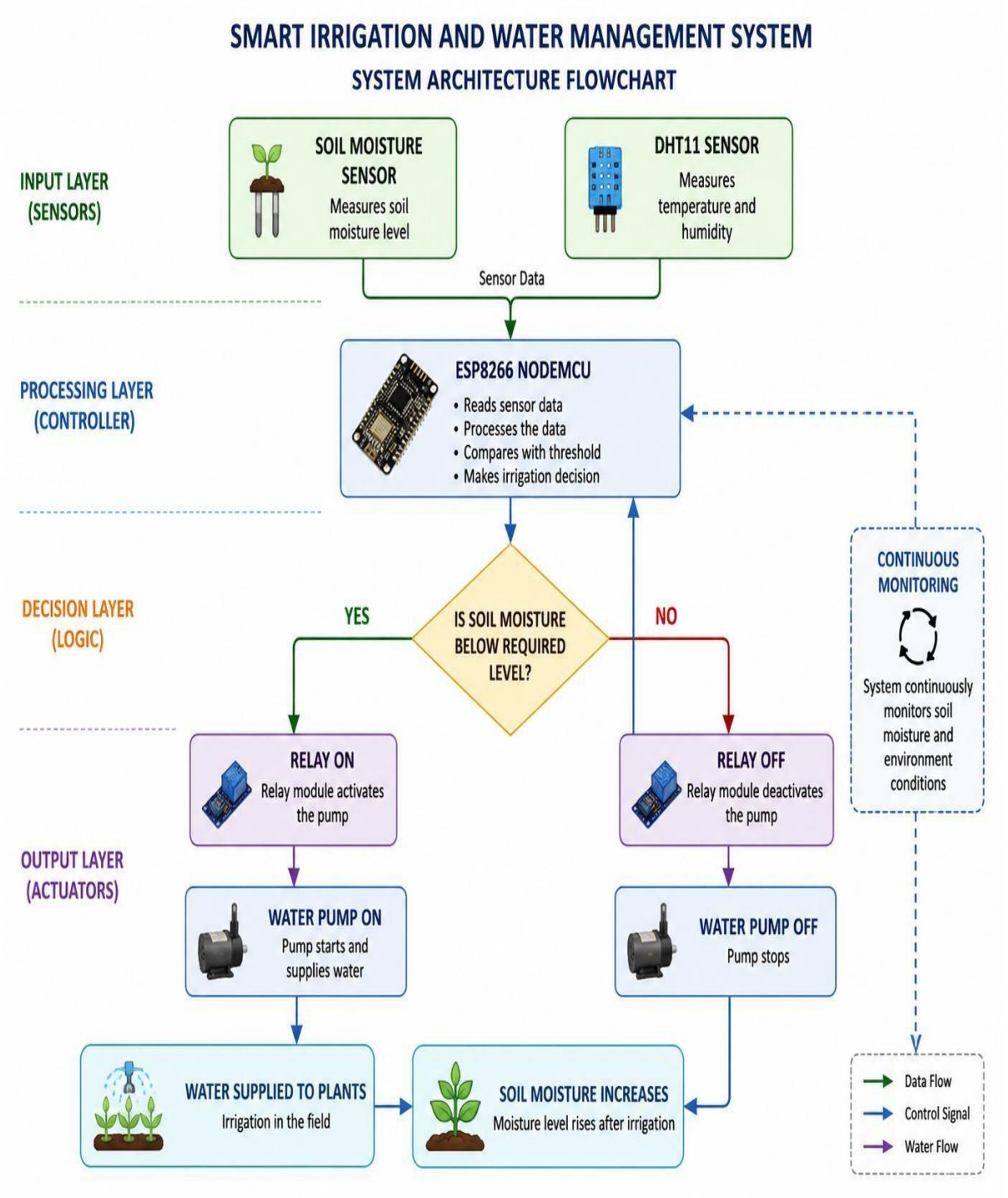
The proposed system is an IoT-based HVAC Temperature and Humidity Monitoring System designed to continuously monitor indoor environmental conditions and improve HVAC performance, occupant comfort, and energy management. The system combines temperature and humidity sensors, an ESP32 microcontroller, Wi-Fi communication, an IoT dashboard, and HVAC control to create a smart and connected monitoring system.

- Real-time Monitoring: Continuously measure room temperature and relative humidity using sensors installed at suitable locations.
- IoT Connectivity: Use an ESP32 microcontroller to collect sensor readings and transmit the data through Wi-Fi.
- Live Dashboard: Display real-time temperature and humidity values on an IoT dashboard for easy monitoring.
- Graphical Visualization: Display temperature and humidity variations using graphs and charts for better analysis.
- Automatic Alerts: Generate alerts when temperature or humidity exceeds predefined safe or comfortable limits.
- HVAC Performance Monitoring: Continuously observe environmental changes to evaluate whether the HVAC system is operating effectively.
- Overcooling Detection: Identify situations where the room temperature falls below the desired range due to excessive cooling.
- Overheating Detection: Detect when the temperature rises above the required comfort level.

- Humidity Monitoring: Identify excessive or insufficient humidity conditions that may affect comfort and indoor environmental quality.
- Automatic HVAC Control: Interface the ESP32 with the HVAC control system to adjust operation according to predefined temperature and humidity conditions.
- Energy Management: Reduce unnecessary HVAC operation by using actual environmental measurements for better operating decisions.
- Data Logging: Store temperature and humidity readings at regular intervals for future analysis.
- Historical Analysis: Compare recorded data over hours, days, or months to identify environmental trends and HVAC operating patterns.
- Remote Monitoring: Allow facility operators to monitor environmental conditions remotely using a smartphone, tablet, or computer.
- Fault Identification: Detect unusual temperature or humidity patterns that may indicate improper HVAC operation or possible system faults.
- Preventive Maintenance Support: Use historical data and abnormal readings to identify potential HVAC problems at an early stage.
- Zone Monitoring: Monitor temperature and humidity in multiple rooms or zones using additional sensor nodes.
- Centralized Monitoring: Combine data from multiple zones into a single IoT dashboard for easier supervision.
- Improved Occupant Comfort: Maintain indoor conditions within a suitable temperature and humidity range.
- Reduced Energy Consumption: Support efficient HVAC operation by minimizing unnecessary cooling or heating.

4. System Architecture

The overall architecture of the system is:



System Flow

Sensors → Microcontroller → Decision Making → Relay → Pump → Irrigation

5. Circuit Table

Component	Pin	ESP8266 NodeMCU
Soil Moisture Sensor	VCC	3V3
Soil Moisture Sensor	GND	GND
Soil Moisture Sensor	AO	A0
DHT11	VCC	3V3
DHT11	GND	GND
DHT11	DATA	D2
Relay Module	VCC	VIN
Relay Module	GND	GND
Relay Module	IN	D3
Relay Module	COM	Pump power supply +
Relay Module	NO	Pump +
Water Pump	−	Pump power supply −

Power Supply

The ESP8266 can be powered through USB. The relay is powered from the appropriate 5V supply through VIN when the board is USB powered.

The water pump should use a suitable power supply according to its rated voltage and current. For a 5V pump, a regulated 5V supply or suitable boost converter should be used.

6. Circuit Design

The circuit consists of five major sections:

6.1 ESP8266 NodeMCU

The ESP8266 is the main controller. It receives sensor data and controls the relay according to the soil moisture condition.

6.2 Soil Moisture Sensor

The soil moisture sensor is connected to the analog input A0.

It provides an analog value representing the moisture condition of the soil.

Soil Sensor AO → ESP8266 A0

6.3 DHT11 Sensor

The DHT11 measures:

- Temperature.
- Relative humidity.

DHT11 DATA → ESP8266 D2

6.4 Relay Module

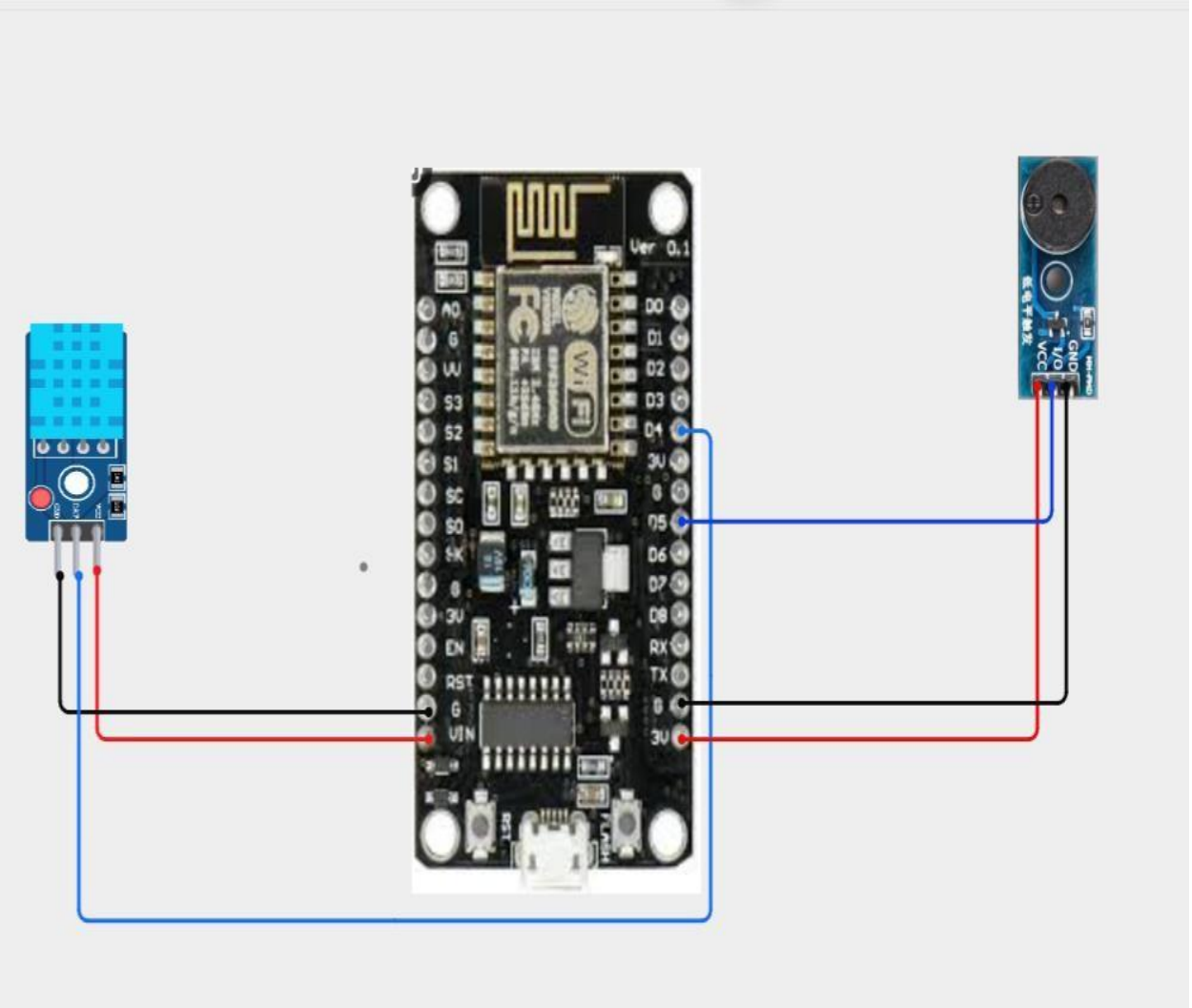
The relay provides electrical switching between the low-power controller and the water pump.

Relay IN → ESP8266 D3

6.5 Water Pump

The water pump supplies water to the plants. It is controlled through the relay and should not be connected directly to an ESP8266 GPIO pin.

CIRCUIT DESIGN:

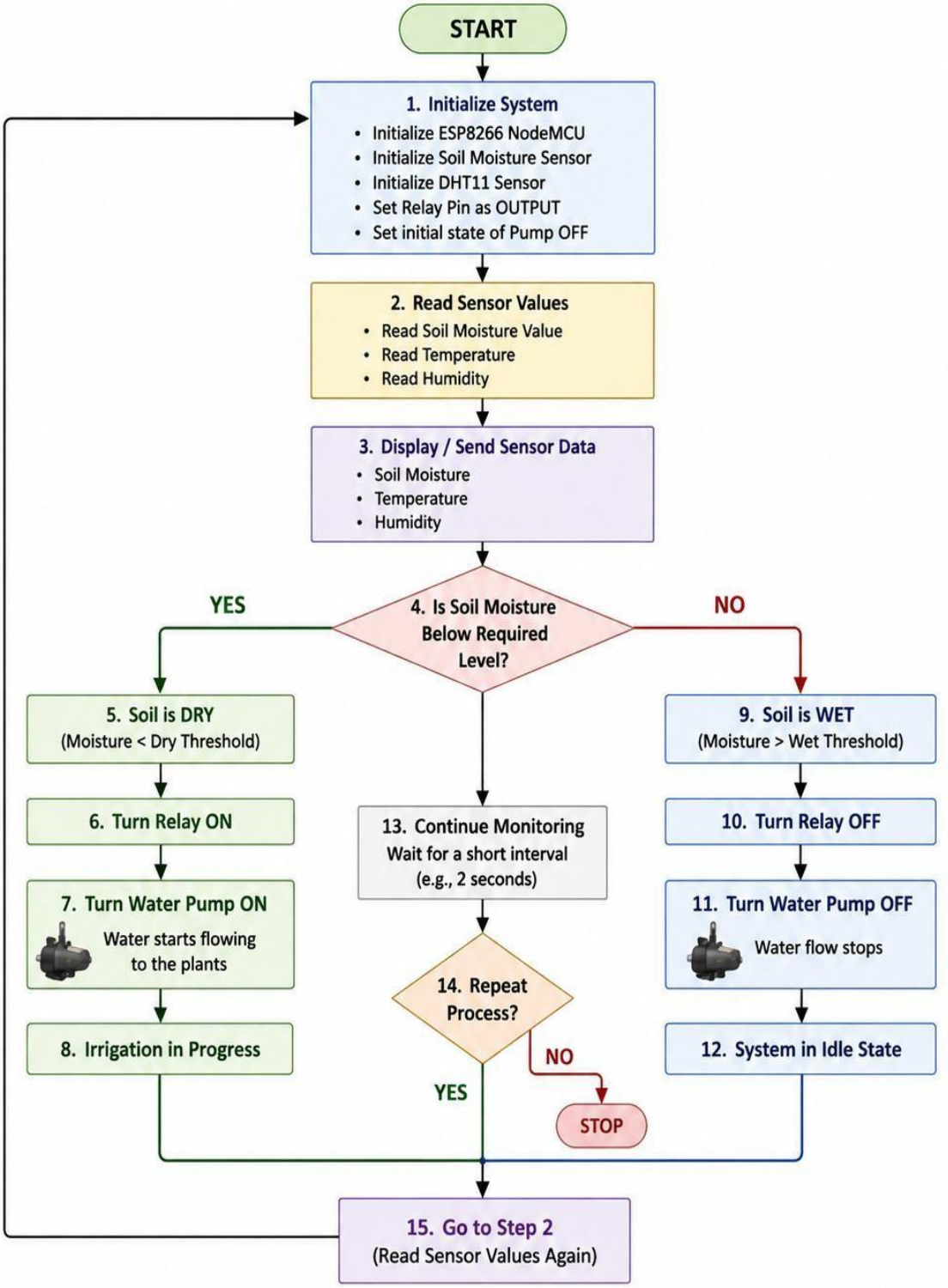


7. Algorithm

- Start the system and initialize the ESP32, temperature and humidity sensor, Wi-Fi, IoT dashboard, display, and HVAC control interface.
- Connect the ESP32 to the available Wi-Fi network.
- Read the current temperature and relative humidity values from the sensor.
- Check whether the sensor data is valid and within the measurable range.
- Display the measured temperature and humidity values on the local display.
- Send the sensor readings to the IoT cloud/dashboard through Wi-Fi.
- Compare the measured temperature with the predefined minimum and maximum temperature limits.
- Compare the measured humidity with the predefined minimum and maximum humidity limits.
- If temperature is higher than the desired range, generate a high-temperature alert and provide a cooling requirement signal.
- If temperature is lower than the desired range, generate a low-temperature alert and reduce or stop cooling operation.
- If humidity exceeds the predefined limit, generate a high-humidity alert and indicate the need for dehumidification or HVAC adjustment.
- If both temperature and humidity are within the desired range, maintain normal HVAC operation.
- Store the temperature and humidity readings with their corresponding time information for historical analysis.
- Continuously monitor the readings and identify unusual changes that may indicate inefficient HVAC operation or possible faults.
- Update the dashboard with the latest sensor values, status, alerts, and graphs.
- Repeat the monitoring process at a predefined time interval.
- If Wi-Fi connectivity is lost, attempt to reconnect automatically and continue local monitoring.

SMART IRRIGATION AND WATER MANAGEMENT SYSTEM

ALGORITHM FLOWCHART



8. Coding

The final working firmware (DHT11 + buzzer + LED alert, standalone version):

```
/*
================================================================
===
  DHT Temperature/Humidity Monitor with ThingSpeak Cloud Upload
  ------------------------------------------------------------------------
  Wiring:
    DHT    VCC  -> 3V3
    DHT    DATA -> GPIO2  (D4)
    DHT    GND  -> G
    Buzzer VCC  -> 3V3
    Buzzer GND  -> G
    Buzzer I/O  -> GPIO14  (D5)

  NOTE: Wi-Fi is required for ThingSpeak upload, so it's enabled here
  (unlike the standalone version). If you see boot instability again
  (blank Serial Monitor after reset), that's the known ESP8266 boot
  power-spike issue — adding a 470-1000uF capacitor across 3V3/G is
  the fix, as discussed earlier.

  Library required (Arduino Library Manager):
    - DHT sensor library (Adafruit)
    - Adafruit Unified Sensor
  (ESP8266WiFi / ESP8266HTTPClient / WiFiClient are built into the
  ESP8266 board package — no separate install needed)

================================================================
=== */


#include <ESP8266WiFi.h>
```

```
#include <ESP8266HTTPClient.h>
#include <WiFiClient.h>
#include <DHT.h>

/* ------------------------ WIFI CONFIG ---------------------------- */
#define WIFI_SSID       "test"
#define WIFI_PASSWORD   "test1234"

/* ------------------------ THINGSPEAK CONFIG ---------------------- */
#define TS_API_KEY      "P4349865UTOWWE3N"
#define TS_SERVER       "http://api.thingspeak.com/update"
#define TS_UPLOAD_INTERVAL_MS  20000UL  // must be >=15000 (ThingSpeak free tier limit)

/* ---------------------- SENSOR/BUZZER CONFIG --------------------*/
#define DHTPIN        2
#define DHTTYPE       DHT11
#define BUZZER_PIN    14

#define BUZZER_ON     LOW
#define BUZZER_OFF    HIGH

#define TEMP_MAX_C        30.0
#define TEMP_MIN_C        16.0
#define HUMIDITY_MAX_PCT  75.0

#define READ_INTERVAL_MS  2000UL
#define BUZZER_BEEP_MS      300UL

DHT dht(DHTPIN, DHTTYPE);
```

```
unsigned long lastReadMs  = 0;
unsigned long lastBeepMs  = 0;
unsigned long lastUploadMs = 0;
bool          buzzerOn     = false;
bool        alarmActive   =   false;
float        lastTempC    =   NAN;
float         lastHumidity = NAN;

void connectWiFi() {
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  Serial.print("Connecting to WiFi");

  unsigned long start = millis();
  while (WiFi.status() != WL_CONNECTED && millis() - start < 15000) {
    delay(300);
    Serial.print(".");
  }

  if (WiFi.status() == WL_CONNECTED) {
    Serial.printf("\nWiFi connected. IP: %s\n", WiFi.localIP().toString().c_str());
  } else {
    Serial.println("\nWiFi connection failed — will retry in loop().");
  }
}

void uploadToThingSpeak(float tempC, float humidityPct) {
  if (WiFi.status() != WL_CONNECTED) {
    Serial.println("[UPLOAD] Skipped — WiFi not connected");
    return;
```

```
  }

  WiFiClient client;
  HTTPClient http;

  String url = String(TS_SERVER) +
               "?api_key=" + TS_API_KEY +
               "&field1=" + String(tempC, 1) +
               "&field2=" + String(humidityPct, 1);

  http.begin(client, url);
  int httpCode = http.GET();

  if (httpCode > 0) {
    String response = http.getString();
    Serial.printf("[UPLOAD] HTTP %d, entry ID: %s\n", httpCode, response.c_str());
  } else {
    Serial.printf("[UPLOAD] Failed, error: %s\n", http.errorToString(httpCode).c_str());
  }

  http.end();
}

void setup() {
  Serial.begin(115200);
  delay(200);

  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, BUZZER_OFF);
```

```
  // Startup beep — confirms buzzer wiring/polarity each boot
  digitalWrite(BUZZER_PIN, BUZZER_ON);
  delay(200);
  digitalWrite(BUZZER_PIN, BUZZER_OFF);

  dht.begin();
  connectWiFi();

  Serial.println("DHT + ThingSpeak monitor ready.");
}

void loop() {
  unsigned long now = millis();

  if (WiFi.status() != WL_CONNECTED) {
    connectWiFi();
  }

  /* --- Read sensor every READ_INTERVAL_MS --- */
  if (now - lastReadMs >= READ_INTERVAL_MS) {
  lastReadMs = now;

    float humidityPct = dht.readHumidity();
    float tempC       = dht.readTemperature();

    if (isnan(humidityPct) || isnan(tempC)) {
      Serial.println("[ERROR] Failed to read from DHT sensor");
      alarmActive = false;
      digitalWrite(BUZZER_PIN, BUZZER_OFF);
      return;
```

```
  }


  lastTempC    = tempC;
  lastHumidity = humidityPct;

  Serial.printf("Temp: %.1f C   Humidity: %.1f %%\n", tempC, humidityPct);

  alarmActive = (tempC >= TEMP_MAX_C) ||
                (tempC <= TEMP_MIN_C) ||
                (humidityPct >= HUMIDITY_MAX_PCT);

  if (alarmActive) {
    Serial.println("  -> ALERT: reading out of safe range!");
  }
}


/* --- Upload to ThingSpeak every TS_UPLOAD_INTERVAL_MS --- */
if (now - lastUploadMs >= TS_UPLOAD_INTERVAL_MS && !isnan(lastTempC)) {
  lastUploadMs = now;
  uploadToThingSpeak(lastTempC, lastHumidity);
}


/* --- Buzzer pattern while alarm is active --- */
if (alarmActive) {
  if (now - lastBeepMs >= BUZZER_BEEP_MS) {
    lastBeepMs = now;
    buzzerOn = !buzzerOn;
    digitalWrite(BUZZER_PIN, buzzerOn ? BUZZER_ON : BUZZER_OFF);
  }
} else if (buzzerOn) {
```

```
    buzzerOn = false;
    digitalWrite(BUZZER_PIN,
    BUZZER_OFF);
  }
}
```

A cloud-connected variant of this firmware was also developed, which re-enables the Wi-Fi radio and adds an HTTP GET call to the ThingSpeak Update API (http://api.thingspeak.com/update) every 20 seconds, posting temperature to Field 1 and humidity to Field 2 of ThingSpeak Channel ID 3455092.

Code Explanation

- A0 reads the soil moisture sensor.
- D2 receives data from the DHT11.
- D3 controls the relay.
- dryThreshold determines when irrigation should start.
- wetThreshold determines when irrigation should stop.
- The DHT11 provides temperature and humidity information.
- The relay controls the water pump.

The threshold values should be calibrated according to the actual soil and sensor used.

9. System Working

During an alert condition, **the** LED remains continuously ON to provide a visual warning. At the same time, the buzzer is activated in pulses at approximately 300 ms intervals, providing an audible indication that the environmental conditions have moved outside the safe range. The controller continues monitoring the sensor rather than stopping the measurement process.

Once the temperature and humidity values return to the predefined safe limits, the NodeMCU automatically exits the alert state. The LED is switched OFF and the buzzer is silenced. This automatic reset allows the system to operate continuously without requiring manual intervention.

For IoT-based monitoring, the NodeMCU connects to a 2.4 GHz Wi-Fi network and periodically sends the latest environmental data to ThingSpeak. The data is uploaded approximately every **20 seconds**, while local sensing continues every 2 seconds. The ThingSpeak channel stores the received temperature and humidity values and displays them using live graphs. This allows environmental conditions to be monitored remotely and helps identify temperature or humidity changes over time.

The complete process therefore follows a continuous cycle of **s**ensing → validation → display → limit comparison → alert generation → cloud transmission → automatic recovery. This makes the system suitable for HVAC monitoring, laboratories, storage areas, classrooms, server rooms, and other environments where maintaining suitable temperature and humidity conditions is important.

10. Bill of Materials

S.No.	Component	Quantity
1	ESP8266 NodeMCU	1
2	Soil Moisture Sensor	1
3	1-Channel Relay Module	1
4	Mini Water Pump	1
5	DHT11 Sensor	1
6	Breadboard	1
7	Jumper Wires	1 set
8	Suitable Pump Power Supply	1

Software Requirements

- Arduino IDE
- ESP8266 Board Package
- DHT Sensor Library
- Cirkit Designer for circuit design/simulation

11. Prototype Implementation

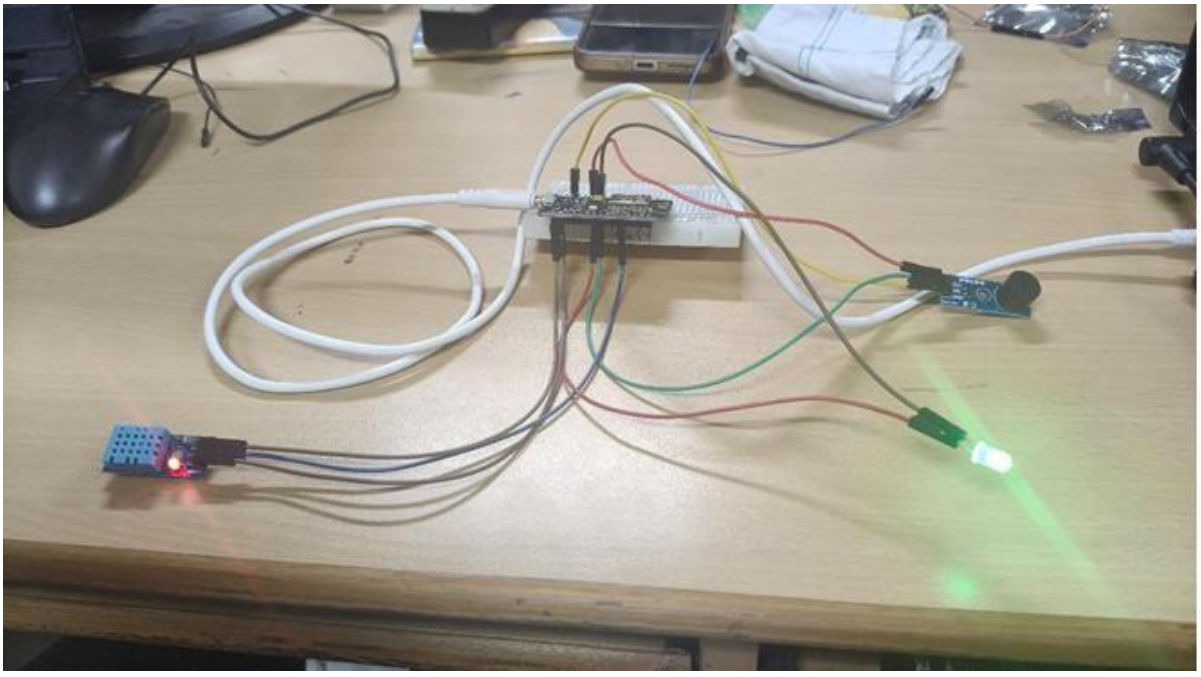
The prototype was designed using an ESP8266 NodeMCU, soil moisture sensor, DHT11 sensor, relay module and mini water pump.

The components were first arranged and connected in Cirkit Designer to develop the circuit design.

The ESP8266 was selected as the main controller because it provides sufficient GPIO and analog input capability and can also support Wi-Fi-based IoT extensions.

The prototype operates automatically according to the soil moisture condition.

Prototype Process



The prototype development begins by defining the required temperature and humidity range based on the selected HVAC application. Suitable components such as an ESP32 development board, temperature and humidity sensor, OLED/LCD display, relay module, power supply, and IoT platform are selected. The temperature and humidity sensor is then connected to the ESP32, while the OLED/LCD display is integrated to show real-time environmental readings locally. The ESP32 is connected to a Wi-Fi network to enable

wireless data transmission. The required program is developed to collect temperature and humidity readings at regular intervals and transmit the data to an IoT dashboard. The dashboard is configured to display real-time values along with graphical representations of temperature and humidity variations.

Predefined temperature and humidity threshold values are established according to the required operating conditions. The system is programmed to generate alerts whenever the measured values exceed the specified limits. A relay or HVAC control interface is connected to the ESP32 to demonstrate automatic HVAC control based on the measured environmental conditions. The prototype is tested under different conditions, including normal temperature, high temperature, low temperature, and high humidity. The sensor readings are compared with a standard thermometer and hygrometer to verify measurement accuracy. The collected data is monitored through the IoT dashboard and analyzed to identify HVAC inefficiencies and possible energy-saving opportunities. Based on the test results, the control limits and system response are optimized. Finally, the complete prototype is assembled with proper enclosure, wiring, sensor placement, and safety precautions, followed by a demonstration of real-time monitoring, alert generation, data logging, and HVAC control.

12. Results & Performance Analysis

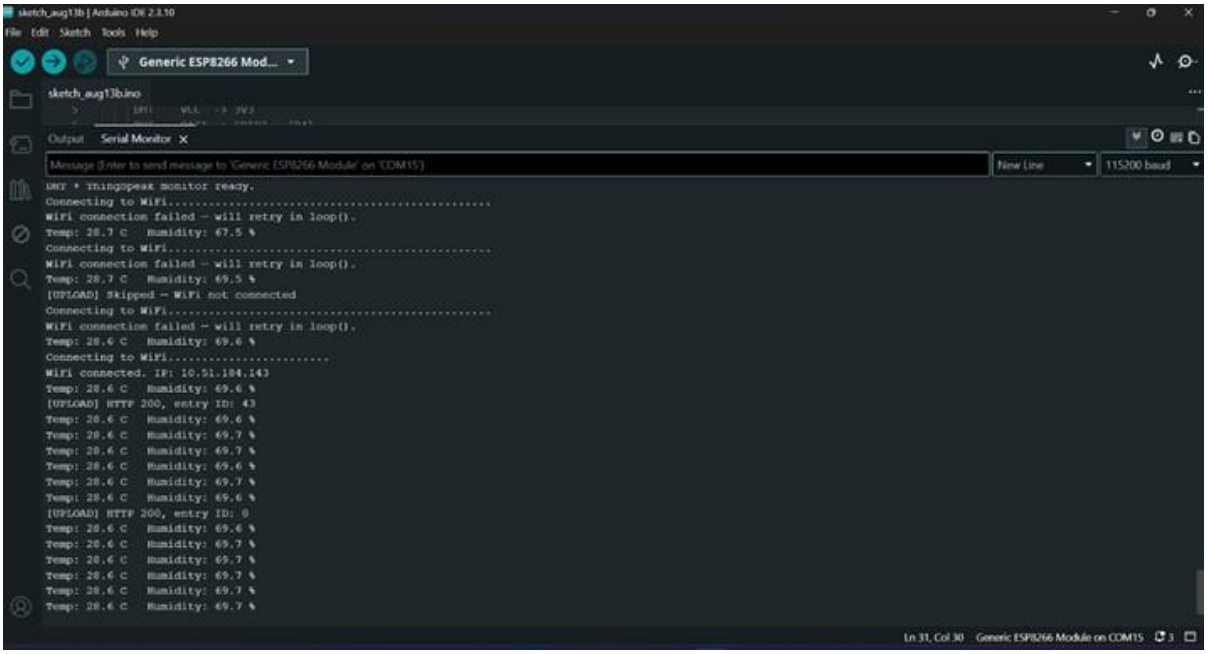
Once the wiring and configuration issues were resolved, the prototype performed successfully as designed. The DHT11 sensor produced consistent temperature and humidity readings at 2-second intervals, with values such as 28.5°C and 71.3% RH observed during indoor testing. The alert logic correctly detected when humidity exceeded the configured 70–75% threshold, displaying the message “ALERT: reading out of safe range!” and activating the LED reliably.

During testing, an unusually high temperature reading of approximately 40°C was observed in an air-conditioned room. This was identified as likely being caused by the sensor being placed too close to the NodeMCU onboard voltage regulator, highlighting the importance of proper sensor placement away from heat-generating components. Wi-Fi connectivity was successfully established after correcting the SSID configuration, and the system obtained an assigned IP address, confirming successful wireless communication.

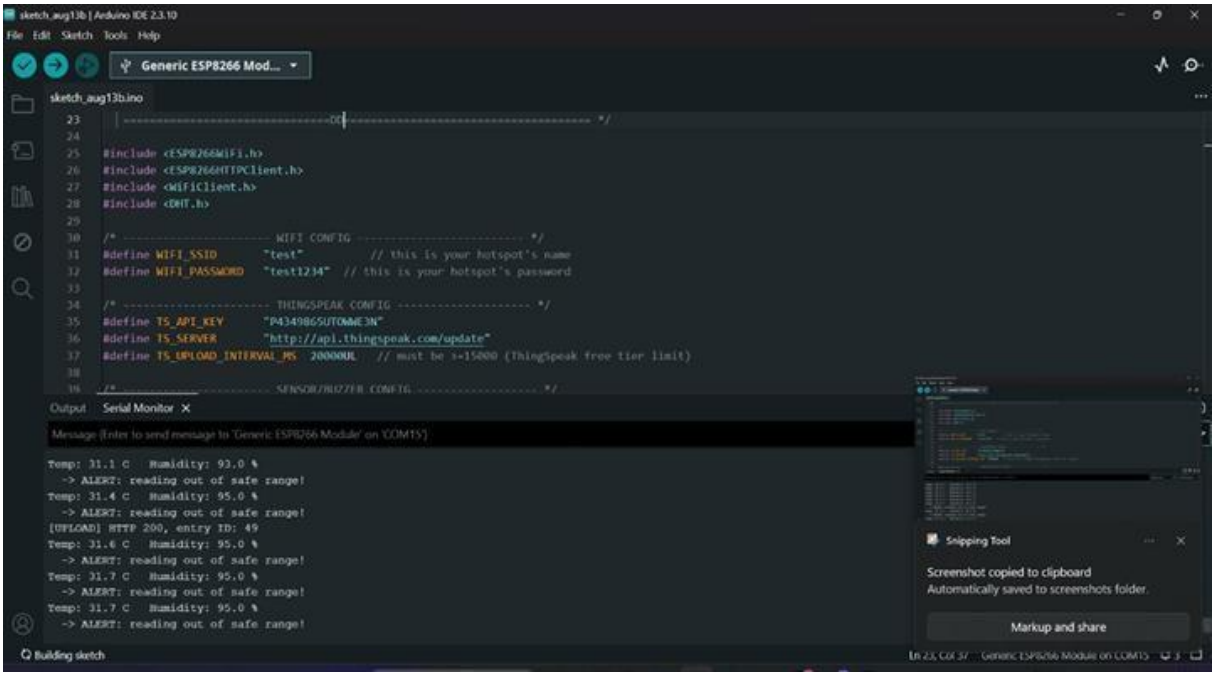
Cloud connectivity through ThingSpeak Channel 3455092 was configured with a 20-second upload interval for remote monitoring. The buzzer alert was not consistently verified and requires further hardware testing to identify any wiring or power-related issues. Overall, the prototype successfully demonstrates temperature and humidity sensing, local alert generation, and Wi-Fi connectivity. Further improvements include confirming stable cloud data transmission, improving power stability, verifying the buzzer, and integrating HVAC control for better energy management and automated operation.

The performance analysis shows that the system is capable of continuously monitoring indoor environmental conditions and responding to changes in humidity and temperature. The sensor data can be used to identify unsuitable conditions and support better HVAC operation. The successful Wi-Fi connection provides the basis for remote monitoring through the IoT platform, while the alert mechanism helps users quickly identify abnormal conditions. The prototype also highlights the importance of accurate sensor placement and stable power supply for reliable measurements. With further testing and improvements, the system can be expanded to monitor multiple rooms, store long-term environmental data, and automatically

control HVAC equipment based on real-time conditions. This can help improve occupant comfort, reduce unnecessary HVAC operation, and support efficient energy management.



Serial Monitor — Wi-Fi connection sequence



Serial Monitor — alert triggers and ThingSpeak upload log